

Analytics Dashboard

A Python-powered portfolio analytics and decision-support dashboard built with Pandas, NumPy, Plotly and Dash.

INPUT Excel transaction data	ENGINE Python + Pandas + NumPy	VISUALS Plotly + Dash	DEPLOYMENT GitHub + Render
--	--	---------------------------------	--------------------------------------

PROJECT PURPOSE

From raw transactions to a decision-ready dashboard

The project transforms transaction-level mutual fund data into a structured analytics layer and presents the results through a professional, interactive web dashboard.

1. DATA Sample Transactions.xlsx Scheme • Type • Units • NAV • Amount • Date	2. PROCESS Clean → normalize → aggregate purchases / redemptions / holdings	3. ANALYZE Returns • XIRR • allocation cash flow • rankings • health	4. PRESENT Dash tabs • tables interactive Plotly charts
---	--	---	--

Core outcome

The result is a reusable analytics application rather than a static Excel report: the same processing pipeline can ingest a transaction dataset, calculate portfolio metrics and render the dashboard automatically.

GitHub repository	github.com/souravsengupta125-dev/mutual-fund-portfolio-dashboard
Live dashboard	https://mutual-fund-portfolio-dashboard.onrender.com

Business problem and analytical approach

The dashboard follows a simple BI principle: convert raw transactional records into metrics that answer portfolio-management questions quickly.

What is being solved

Manual portfolio tracking spreads information across transactions, current holdings and calculations. The application centralizes these into one analytical view.

What the user gets

A four-tab dashboard for portfolio overview, current holdings, performance and fund health, with interactive tables and charts.

Input data model

Field	Role in the model
Scheme Name	Identifies the mutual fund / holding.
Transaction Type	Separates PURCHASE and REDEEM cash flows.
Units	Tracks units acquired or redeemed.
NAV	Stores the transaction NAV used with units.
Amount	Drives investment and redemption cash-flow analysis.
Date	Provides the time dimension for performance and XIRR.

Processing pipeline

- Read the Excel transaction dataset and normalize column names and text values.
- Convert dates and numeric fields into analysis-ready types; remove blank / duplicate rows and handle invalid numeric values.
- Aggregate transactions into a master portfolio view containing purchased units, redeemed units, current units, invested amount, redeemed amount, cost value, current value, profit and return.
- Derive holding weight, profit contribution, gain/loss status and ranked views for holdings, gainers, losers and XIRR.

The anonymized demonstration dataset uses generic fund names such as Mutual Fund 1, Mutual Fund 2, etc., so the project can be shared publicly without exposing a personal portfolio.

Core calculations and portfolio intelligence

The analytical layer sits between the Excel input and the Dash presentation. This separation makes the dashboard easier to understand, test and extend.

Analytic	How it is derived	Decision value
Current holdings	Aggregate purchased units less redeemed units and combine with current NAV / holding data.	Shows what is currently held and its market value.
Profit / Return	Compare current value with cost value and calculate percentage return.	Highlights gain / loss and relative performance.
Fund XIRR	Build dated cash flows, treat purchases as outflows, redemptions as inflows, append current value as the terminal inflow, then solve for the rate.	Measures annualized performance for irregular cash flows.
Asset allocation	Map each fund to an asset class and aggregate current value by class.	Shows portfolio composition.
Market-cap allocation	Map funds to large / mid / small / flexi-cap categories and aggregate current value.	Shows equity style exposure.
Monthly cash flow	Group purchase and redemption amounts by month and derive net investment.	Shows investment rhythm and liquidity movement.

XIRR implementation

The application creates a dated cash-flow series. PURCHASE amounts are negative cash flows, REDEEM amounts are positive cash flows, and the current portfolio value is appended as a final positive cash flow dated today. The annualized rate is solved numerically using SciPy's brentq root finder.

$$NPV(r) = \sum \text{Cash Flow} / (1 + r)^{(\text{days from first cash flow} / 365)}$$

The same logic is applied at portfolio level and fund level, provided the fund has both positive and negative cash flows. The calculated fund XIRRs are rounded for presentation.

Classification layer

Asset class	Equity, Debt, Hybrid and Multi Asset mappings are used to build allocation summaries.	Market cap	Large Cap, Mid Cap, Small Cap, Flexi Cap and related categories are mapped for allocation analysis.	AMC	The application derives an AMC field from the scheme name and summarizes current value by AMC.
--------------------	---	-------------------	---	------------	--

Four analytical views, one consistent interface

Plotly Dash is used as the presentation layer. The interface uses reusable cards, responsive Plotly charts and interactive DataTables to keep the dashboard compact and professional.

High-level portfolio KPIs, allocation views, top holdings and performance rankings.	Current units, NAV, current value, profit, return % and XIRR with native sorting/filtering.	Portfolio growth plus monthly purchase and redemption views.	XIRR-based health classification and action-oriented fund review.

Visualization layer

Allocation	Donut charts for asset allocation and market-cap allocation.	Performance	Line and bar charts for portfolio growth, returns and XIRR rankings.
Cash flow	Monthly purchase, redemption and net-investment charts.	Tables	Dash DataTables provide native sorting, filtering and pagination.

UI engineering details

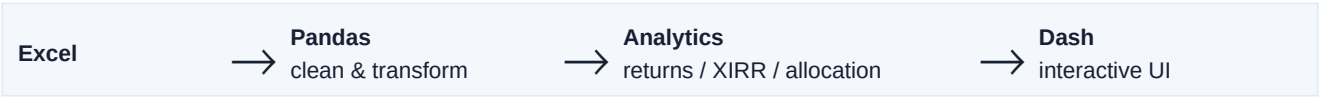
- Dash Bootstrap Components provide a clean responsive foundation.
- A centralized theme defines background, card, typography, border and status colors.
- Reusable section-card and KPI-card patterns keep spacing and visual hierarchy consistent.
- Interactive tables use native sorting, filtering and pagination rather than static HTML tables.
- The application is designed to run both locally and as a hosted Dash web service.

Technical architecture and deployment

The project is intentionally lightweight: an Excel file is the source, Python performs the transformation and analytics, and Dash renders the result.

Layer	Technology	Responsibility
Data source	Excel / OpenPyXL	Read anonymized transaction records.
Data processing	Pandas + NumPy	Cleaning, transformation, aggregation and calculations.
Numerical solving	SciPy	Numerical solution of XIRR using brentq.
Visualization	Plotly	Interactive charts and analytical visualizations.
Application	Plotly Dash	Tabs, KPI cards, tables and web layout.
Styling	Dash Bootstrap Components	Responsive visual framework and presentation.
Version control	GitHub	Source code, sample data, documentation and screenshots.
Hosting	Render + Gunicorn	Public web deployment of the Dash application.

Runtime flow



Repository structure

```
mutual-fund-portfolio-dashboard/  
■■■ dashboard.py  
■■■ sample_transactions.xlsx  
■■■ requirements.txt  
■■■ .gitignore  
■■■ README.md  
■■■ overview.png  
■■■ holdings.png  
■■■ performance.png  
■■■ fund_health.png
```

Deployment model

- GitHub is the source-of-truth repository for the project.
- Render is connected to the repository and automatically deploys changes.
- Gunicorn serves the Dash application in the hosted environment.
- The public deployment provides a recruiter-friendly live demonstration without requiring a local Python setup.

What this project demonstrates

The strongest value of the project is not the subject matter alone; it is the end-to-end analytics workflow demonstrated through a working application.

Data engineering

Structured raw transaction data, standardized fields, handled data quality issues and built an analysis-ready master dataset.

Financial analytics

Implemented holdings, profit, return, cash-flow and XIRR calculations at portfolio and fund level.

BI / visualization

Translated analytical outputs into KPI cards, interactive tables, allocation views and time-series charts.

Application development

Built a multi-tab Dash application with reusable UI components and consistent visual styling.

Deployment

Moved the application from local execution to a public GitHub + Render deployment.

Documentation

Created an anonymized, reproducible project suitable for public demonstration and portfolio use.

How to demonstrate it in an interview

Problem → Data → Logic → Dashboard → Deployment. Start with the business problem, explain how transaction data becomes a portfolio model, show one or two important calculations such as XIRR, demonstrate the four dashboard tabs and finish by opening the live Render deployment.

Key technical talking points

- Why transaction-level cash flows are more useful than a static current-holdings snapshot.
- How purchases, redemptions and current value are converted into dated cash flows for XIRR.
- Why Pandas is used as the analytical layer and Plotly Dash as the presentation layer.
- How asset-class and market-cap mappings create portfolio-level allocation analytics.
- How GitHub and Render turn a local analytical script into a shareable web application.

Public demonstration assets

Asset	Purpose
GitHub repository	Source code, README, requirements, anonymized Excel data and screenshots.
Live Render dashboard	Interactive demonstration for recruiters and interviewers.
This PDF	Concise technical and conceptual case study for CVs, portfolios and project submissions.

Data note: The public demonstration uses anonymized sample transactions and generic fund names. It is intended as a technical / analytics portfolio project and not as investment advice.